

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN
VIỆN TRÍ TUỆ NHÂN TẠO



BÁO CÁO THỰC HÀNH **HỌC MÁY**

Đề tài:

**HỆ THỐNG PHÁT HIỆN VƯỢT ĐÈN ĐỎ VÀ
PHÂN TÍCH MẬT ĐỘ GIAO THÔNG ĐỂ ĐIỀU
CHỈNH ĐÈN**

Nhóm sinh viên thực hiện:

1. Nguyễn Duy Hưng - MSSV: 24022349
2. Nguyễn Cảnh Hào - MSSV: 24022319
3. Lương Quang Hoàng - MSSV: 24022337
4. Nguyễn Duy Khánh - MSSV: 24022361

Tháng 6 Năm 2026

MỤC LỤC

1	TỔNG QUAN ĐỀ TÀI	2
1.1	Bối cảnh và động lực nghiên cứu	2
1.2	Tổng quan các nghiên cứu liên quan	2
1.3	Hướng tiếp cận và đóng góp của đề tài	2
2	CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG	3
2.1	Phát hiện đối tượng với kiến trúc YOLOv8	3
2.2	Theo dõi đa đối tượng động học với ByteTrack	3
2.3	Học máy Vectơ hỗ trợ (SVM) trong phân loại không gian	4
2.4	Kỹ thuật phân tích quang học và hình học tính toán	4
3	Phân tích và thiết kế hệ thống	6
3.1	Yêu cầu hệ thống	6
3.2	Kiến trúc tổng thể và luồng dữ liệu	6
3.3	Thiết kế chi tiết các module lõi	8
3.4	Thiết kế module lưu trữ và kết xuất dữ liệu	10
3.5	Thiết kế giao diện trực quan (Overlay UI)	12
4	TRIỂN KHAI HỆ THỐNG	13
4.1	Môi trường phần mềm và thư viện sử dụng	13
4.2	Xây dựng và tiền xử lý tập dữ liệu	14
4.3	Huấn luyện các mô hình phân loại SVM	16
4.4	Cài đặt pipeline xử lý chính (<code>main.py</code>)	17
4.5	Thiết kế cấu trúc tham số toàn cục (<code>config.yaml</code>)	18
5	Đánh Giá	19
5.1	Kết quả nhận diện đèn giao thông	19
5.2	Kết quả phát hiện vi phạm	21
5.3	Kết quả module điều khiển tín hiệu thích ứng	22
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	24
6.1	Kết luận	24
6.2	Những kết quả đạt được	24
6.3	Hạn chế của hệ thống	25
6.4	Hướng phát triển tương lai	25
	Tài liệu tham khảo	25

1 TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh và động lực nghiên cứu

Vượt đèn đỏ là nguyên nhân cốt lõi gây mất an toàn tại các nút giao đô thị. Việc giám sát thủ công hiện nay bộc lộ rõ hạn chế về chi phí, độ trễ và khả năng mở rộng quy mô. Dù công nghệ Học sâu (Deep Learning) và Thị giác máy tính (Computer Vision) đã phát triển mạnh mẽ, các ứng dụng hiện tại chủ yếu vẫn mang tính phân mảnh. Đề tài này được thực hiện với động lực chuyển đổi mô hình giám sát sang dạng tự động, khép kín: không chỉ phát hiện vi phạm và lưu trữ bằng chứng số, mà còn tái sử dụng thông số mật độ phương tiện để điều khiển luồng giao thông thích ứng.

1.2 Tổng quan các nghiên cứu liên quan

- **Hệ thống truyền thống:** Chủ yếu dùng cảm biến vòng từ hoặc hồng ngoại. Nhược điểm lớn nhất là chi phí bảo trì cao và chỉ ghi nhận được sự hiện diện phương tiện tại một tọa độ tĩnh thay vì phân tích toàn bộ quỹ đạo động học.
- **Hệ thống dựa trên Học sâu:** Đã ứng dụng thành công các mô hình như YOLO kết hợp thuật toán bám vết (DeepSORT, ByteTrack) và các bộ phân loại Học máy (SVM) để theo dõi phương tiện và đọc trạng thái đèn tín hiệu.
- **Hạn chế tồn đọng:** Phần lớn các giải pháp chưa tạo thành một đường ống (pipeline) hợp nhất. Các thuật toán dễ bị nhiễu quang học (phơi sáng, thay đổi góc độ). Đặc biệt, hệ thống thiếu cơ chế tự động trích xuất video bằng chứng pháp lý và chưa tích hợp được khả năng điều khiển pha đèn thông minh.

1.3 Hướng tiếp cận và đóng góp của đề tài

Trước khi triển khai, nhóm xem xét ba hướng tiếp cận chính và lựa chọn hướng phù hợp nhất:

- **Cảm biến vật lý truyền thống:** Chi phí lắp đặt cao, chỉ phát hiện sự hiện diện tại một điểm cố định, không lưu được bằng chứng hình ảnh.
- **Xử lý ảnh truyền thống:** Không cần dữ liệu huấn luyện nhưng nhạy cảm với thay đổi ánh sáng, không bám vết được phương tiện qua nhiều khung hình.
- **Học sâu kết hợp Học máy (hướng đề xuất):** YOLOv8 đạt tốc độ thời gian thực, ByteTrack bám vết ổn định không cần Re-ID, SVM phân loại nhanh với đặc trưng thấp chiều. Hướng này được chọn vì cân bằng tốt giữa độ chính xác, tốc độ và khả năng mở rộng.

Từ hướng tiếp cận đề xuất, đề tài xây dựng một kiến trúc phần mềm giám sát giao thông toàn diện (End-to-End) với 6 đóng góp cốt lõi:

1. **Bám vết thời gian thực:** Tích hợp YOLOv8 và ByteTrack để cấp phát và duy trì định danh (ID) bền vững cho phương tiện qua các khung hình.

2. **Nhận diện đèn tín hiệu đa tầng:** Kết hợp YOLOv8 fine-tuned, SVM và cơ chế dự phòng HSV để phân loại trạng thái đèn với độ tin cậy cao trước nhiều môi trường.
3. **Logic phát hiện vi phạm:** Ứng dụng tích có hướng (Cross Product) để kiểm tra giao cắt vạch dừng và Point-in-Polygon để loại bỏ chính xác ngoại lệ rẽ phải hợp lệ.
4. **Trích xuất bằng chứng tự động:** Ring Buffer tự động cắt và đóng gói video clip trước và sau mốc thời gian vi phạm.
5. **Điều khiển đèn thích ứng:** SVM phân loại mật độ xe vi mô để chủ động can thiệp chu kỳ đèn, tối ưu năng lực giải phóng phương tiện tại nút giao.
6. **Giao diện giám sát (HUD):** Overlay UI hiển thị trực quan bounding box, trạng thái đèn, cảnh báo vi phạm và thông số mật độ theo thời gian thực.

2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG

2.1 Phát hiện đối tượng với kiến trúc YOLOv8

YOLOv8 là kiến trúc mạng nơ-ron tích chập (CNN) hiện đại, hoạt động dựa trên cơ chế không mỏ neo (anchor-free) và phân tách luồng dự đoán (Decoupled Head). Việc loại bỏ các mỏ neo tĩnh giúp triệt tiêu bước tinh chỉnh thủ công, đồng thời hàm mất mát *Distribution Focal Loss* (DFL) tối ưu hóa sự cân bằng giữa tốc độ và độ chính xác trong thời gian thực.

Hệ thống sử dụng biến thể **YOLOv8s (Small)** cho cả hai mục đích, được cấu hình qua hai bộ trọng số độc lập trong `config.yaml`:

- **Phát hiện phương tiện** (`vehicle_model: yolov8s.pt`): Sử dụng trọng số COCO, áp dụng bộ lọc tĩnh để chỉ trích xuất 4 lớp phương tiện giao thông: **car** (ID 2), **motorcycle** (ID 3), **bus** (ID 5) và **truck** (ID 7). Ngưỡng confidence là 0,25 và NMS IoU là 0,45.
- **Phát hiện đèn giao thông** (`traffic_light_model: yolov8s.pt`): Mô hình được fine-tune trên bộ dữ liệu chuyên biệt để phân loại 4 trạng thái: **green**, **red**, **yellow** và **off**. Ngưỡng confidence được đặt thấp ở mức 0,05 để tối đa hóa khả năng phát hiện đèn trong điều kiện ánh sáng đa dạng.

2.2 Theo dõi đa đối tượng động học với ByteTrack

ByteTrack giải quyết bài toán theo dõi đa đối tượng (MOT) bằng phương pháp *tracking-by-detection*. Điểm đột phá của thuật toán là cơ chế liên kết hai bước: thay vì loại bỏ các hộp bao có độ tin cậy thấp, ByteTrack tái sử dụng chúng để khớp nối với các quỹ đạo bị đứt gãy thông qua bộ lọc Kalman và thuật toán Hungarian, giúp hạn chế hiện tượng trao đổi định danh (ID switching) khi phương tiện bị che khuất.

Module bám vết (`tracker.py`) tích hợp ByteTrack thông qua API `model.track()` của Ultralytics, tránh việc biên dịch thư viện C++ trên Windows, và bổ sung ba kỹ thuật hiệu chỉnh phù hợp với đặc thù giao thông đô thị:

- **Bộ đệm quỹ đạo (Track Buffer):** Lưu giữ các thực thể mất dấu trong tối đa 60 khung hình (`track_buffer: 60`), tạo khoảng trễ để phục hồi quỹ đạo khi xe xuất hiện trở lại sau khi bị che khuất.
- **Ánh xạ định danh (ID Mapping):** Bảng băm `_raw_to_compact` quy đổi ID thô của ByteTrack (không liên tục, giá trị lớn) thành chuỗi số nguyên liên tục bắt đầu từ 1, giúp log JSON gọn gàng và hiển thị trực quan trên giao diện.
- **Giãn nở bao hình (Box Expansion):** Riêng với nhãn `motorcycle`, hộp bao được mở rộng lên phía trên 60% theo chiều cao ($y_{\min}^{\text{new}} = y_{\min} - 0,6 \times (y_{\max} - y_{\min})$), đảm bảo vùng crop bằng chứng bao trùm cả người điều khiển, không chỉ khung gầm xe.

2.3 Học máy Vectơ hỗ trợ (SVM) trong phân loại không gian

2.3.1 Nguyên lý tối ưu hóa siêu phẳng

Thuật toán Vectơ hỗ trợ (SVM) xây dựng biên giới quyết định bằng cách tìm kiếm một siêu phẳng phân tách $\mathbf{w}^T \mathbf{x} + b = 0$ trong không gian đặc trưng, nhằm cực đại hóa khoảng cách lề (margin) tới các điểm dữ liệu gần nhất của mỗi lớp (support vectors). Đối với bài toán phân loại đa lớp, hệ thống sử dụng chiến lược Một-đối-Một (One-vs-One - OVO), xây dựng $\binom{k}{2}$ bộ phân loại nhị phân và kết xuất nhãn cuối cùng thông qua cơ chế bầu chọn đa số (majority voting).

Để xử lý tính phi tuyến của phân bố dữ liệu giao thông, SVM áp dụng kỹ thuật hàm nhân thông qua hàm Cơ sở Bán kính RBF (Radial Basis Function):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (1)$$

2.3.2 Khai thác mô hình trong hệ thống

Hệ thống triển khai hai bộ phân loại SVM độc lập. Toàn bộ dữ liệu đầu vào được chuẩn hóa thông qua `StandardScaler` và thiết lập siêu tham số `class_weight='balanced'` để bù đắp sự chênh lệch tần suất giữa các lớp:

- **Mô hình phân loại tín hiệu đèn (svm_traffic_light.pkl):** Xử lý vector đặc trưng quang học 27 chiều gồm các giá trị thống kê phân bố màu sắc toàn cục và theo vùng không gian của ảnh crop đèn. Đầu ra gồm 4 trạng thái: `green`, `red`, `yellow`, `off` với hệ số phạt $C = 1,0$.
- **Mô hình phân loại mật độ giao thông (svm_traffic_density.pkl):** Xử lý vector đặc trưng 5 chiều gồm `motorcycle_count`, `car_count`, `stopped_vehicles`, `pcu_load` và `average_speed`. Mô hình phân loại thành 3 mức mật độ (*Low/Empty*, *Medium/Normal*, *High/Congested*) với hệ số phạt $C = 2,0$.

2.4 Kỹ thuật phân tích quang học và hình học tính toán

2.4.1 Không gian màu HSV và bộ lọc dự phòng

Hệ màu HSV (Hue, Saturation, Value) phân tách phổ màu khỏi cường độ chiếu sáng, thể hiện sự ổn định vượt trội so với hệ RGB dưới biến động của quang học tự nhiên. Hệ thống thiết lập các dải giá trị kênh H tĩnh: vùng Đỏ ($0^\circ - 9^\circ$ và $160^\circ - 180^\circ$), vùng Vàng ($10^\circ - 25^\circ$), và vùng Xanh ($35^\circ - 90^\circ$). Cơ chế này hoạt động như một hệ thống dự phòng

(Hybrid Fallback), cho phép hiệu chỉnh trực tiếp trạng thái đèn khi mô hình mạng nơ-ron không hội tụ do ảnh đầu vào bị mờ nhòe hoặc sai lệch phơi sáng.

2.4.2 Phân tích giao cắt bằng Tích có hướng 2D

Hệ thống loại bỏ phép so sánh tọa độ Y thuần túy để áp dụng đại lượng vô hướng của tích có hướng trong không gian 2D (2D pseudo-cross product). Điểm chiếu tham chiếu (reference point) $P(x_P, y_P)$ được gán tại vị trí đáy trung tâm (bottom-center) của hộp bao, phản ánh chuẩn xác vị trí bánh xe tiếp xúc mặt đường.

Với đoạn vạch dừng $\overrightarrow{AB}$, vị trí tương đối của điểm P được biểu diễn qua hàm đại số:

$$f(P, A, B) = (x_B - x_A)(y_P - y_A) - (y_B - y_A)(x_P - x_A) \quad (2)$$

Trạng thái vi phạm được hệ thống xác nhận khi và chỉ khi quỹ đạo di chuyển từ thời điểm $t - 1$ đến t thỏa mãn đồng thời hệ điều kiện giao cắt đoạn thẳng:

$$\begin{cases} f(P_{t-1}, A, B) \cdot f(P_t, A, B) \leq 0 \\ f(A, P_{t-1}, P_t) \cdot f(B, P_{t-1}, P_t) \leq 0 \end{cases} \quad (3)$$

Ràng buộc kiểm tra chéo này loại trừ tuyệt đối các sai số dương tính giả (false positives) khi quỹ đạo phương tiện chỉ cắt qua phần tuyến tính kéo dài vô hình của vạch dừng trên mặt phẳng camera.

2.4.3 Thuật toán Point-in-Polygon (PiP)

Hệ thống thiết lập các không gian đa giác lồng để định nghĩa các khu vực ưu tiên phi tuyến, tiêu biểu là vùng rẽ phải hợp lệ (*Right-turn zone*). Bằng cách áp dụng thuật toán phóng tia (Ray-casting) thông qua hàm `cv2.pointPolygonTest`, hệ thống liên tục kiểm tra vị trí tọa độ P_t . Nếu phương tiện tiến nhập vào không gian đa giác này, toàn bộ cờ vi phạm cấp phát trước đó sẽ bị hệ thống chủ động thu hồi.

2.4.4 Đơn vị xe con quy đổi (PCU)

Tải lượng bề mặt thực tế tại khu vực đếm xe (*Queue ROI*) được mô hình hóa theo đại lượng Đơn vị xe con quy đổi (PCU). Hệ số PCU phản ánh động lực học và diện tích chiếm dụng mặt đường tương đối của từng loại phương tiện:

$$PCU_{load} = 0.3 \cdot N_{motorcycle} + 1.0 \cdot (N_{car} + N_{bus} + N_{truck}) \quad (4)$$

Chỉ số tích chập này cung cấp tham số định lượng nhất quán để mô hình SVM đánh giá chính xác mức độ bão hòa của dòng giao thông hỗn hợp.

2.4.5 Cấu trúc Hàng đợi Vòng (Ring Buffer) trích xuất luồng sự kiện

Nhằm tối ưu hóa tài nguyên ghi nhớ luồng video liên tục và hỗ trợ kết xuất bằng chứng pháp lý, hệ thống triển khai cấu trúc dữ liệu Hàng đợi Vòng (Ring Buffer) thông qua lớp `collections.deque`. Trong trạng thái giám sát, bộ đệm liên tục ghi đè các khung hình, bảo lưu một cửa sổ thời gian quá khứ cố định (`clip_before_sec`). Khi tín hiệu vi phạm được kích hoạt, tiến trình sẽ giữ lại luồng lịch sử, thu thập bổ sung luồng tương lai (`clip_after_sec`) và ghép nối thành một tệp video `.mp4` hoàn chỉnh, minh họa trọn vẹn diễn biến trước và sau khi hành vi cắt vạch xảy ra.

3 Phân tích và thiết kế hệ thống

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

Hệ thống được thiết kế để giải quyết đồng thời bài toán giám sát an toàn và điều tiết giao thông thông qua 5 nhóm tính năng:

- **Bám vết phương tiện:** Cấp phát định danh (track ID) và liên kết quỹ đạo liên tục cho ô tô, xe máy, xe tải, xe buýt.
- **Đọc tín hiệu đèn:** Phân loại trạng thái (RED, GREEN, YELLOW) thông qua mạng SVM, tích hợp màng lọc không gian màu HSV làm cơ chế dự phòng (fallback) chống nhiễu quang học.
- **Lập hồ sơ vi phạm:** Tự động trích xuất metadata (ID, tọa độ, timestamp, độ tin cậy) ra tệp JSON, đồng thời lưu trữ hình ảnh và video clip cắt tự động qua bộ đệm vòng (Ring Buffer).
- **Điều khiển đèn thích ứng:** Tự động thay đổi chu kỳ đèn (rút ngắn hoặc kéo dài) dựa trên mật độ phương tiện thực tế nội suy tại vùng chờ (*Queue ROI*).
- **Đa chế độ vận hành:** Hỗ trợ 3 tham số CLI: `violation` (chỉ bắt lỗi), `adaptive` (chỉ điều khiển) và `compare` (thực thi và đối chiếu song song).

3.1.2 Yêu cầu phi chức năng

- **Thời gian thực:** Pipeline xử lý phải duy trì ≥ 15 FPS trên luồng video 720p với sự hỗ trợ của GPU, không sinh độ trễ tích lũy.
- **Độ đặc hiệu cao:** Triệt tiêu Dương tính giả (False Positive) bằng tổ hợp logic: vector hướng di chuyển, thuật toán Tích có hướng (Cross-product) khi cắt vạch, và Point-in-Polygon (PiP) để an xá xe rẽ phải.
- **Tối ưu bộ nhớ:** Quản trị chặt chẽ vòng đời biến thông qua `track_buffer`, ngăn chặn rò rỉ RAM/VRAM khi vận hành dài hạn.

3.2 Kiến trúc tổng thể và luồng dữ liệu

3.2.1 Sơ đồ luồng xử lý (Pipeline Architecture)

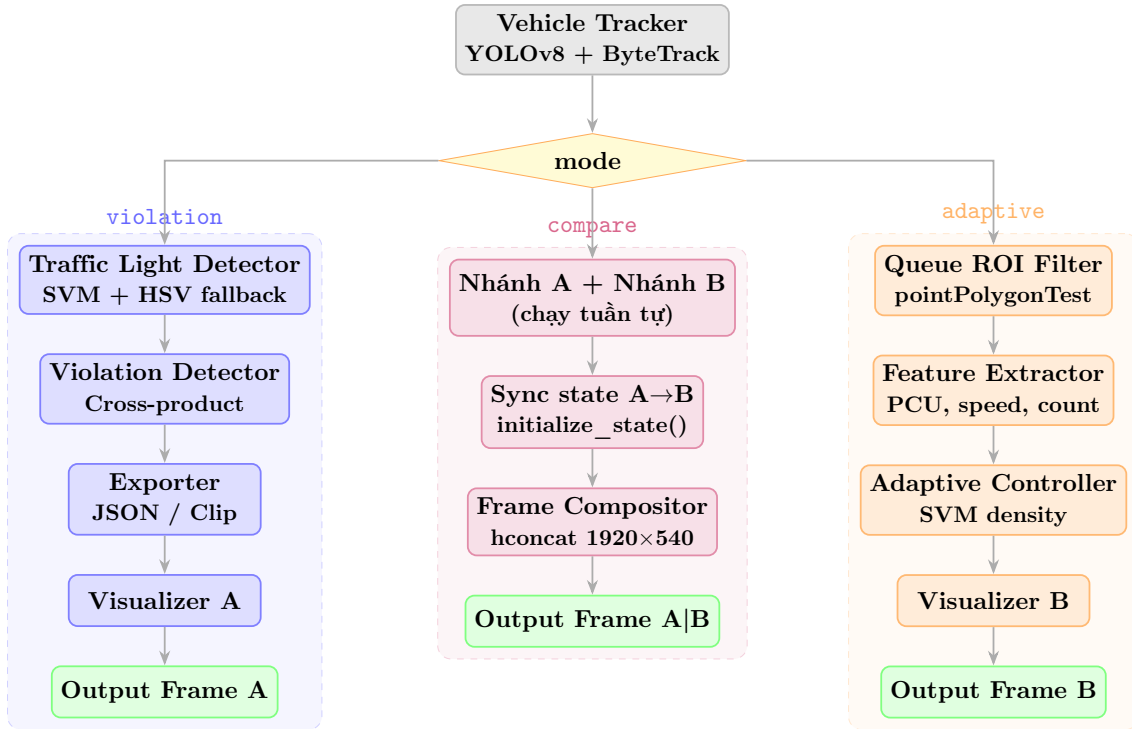
Hệ thống tuân thủ kiến trúc đường ống tuyến tính (Linear Pipeline), điều phối tập trung qua hàm `run_pipeline()`. Khung hình từ đầu vào (Video Input) đi qua bộ Bám vết (Vehicle Tracker), sau đó phân rã thành hai nhánh xử lý song song: **Nhánh Vi phạm** (đối chiếu đèn và quỹ đạo) và **Nhánh Thích ứng** (nội suy mật độ điều khiển đèn). Kết quả cuối cùng được mạ lớp đồ họa (Visualizer) và ghi đĩa.

3.2.2 Mô tả chi tiết 5 phân đoạn xử lý

1. **Đọc dữ liệu (Frame Generator):** `VideoStreamHandler` nội suy video về độ phân giải chuẩn. Thay vì nạp toàn cục, hệ thống dùng cơ chế `yield` để sinh từng khung hình (NumPy Array), tiết kiệm tối đa RAM.
2. **Động học (Vehicle Tracker):** YOLOv8 và ByteTrack cấp phát định danh và tọa độ. *Điểm nhấn kỹ thuật: Riêng với nhãn xe máy, trục Y của hộp bao (bounding box) tự động mở rộng lên trên 60% nhằm thu thập trọn vẹn nhân thể người lái, phục vụ tính pháp lý của bằng chứng.*
3. **Nhánh A - Xử lý vi phạm:** `TrafficLightDetector` phân loại đèn qua mô hình SVM (có fallback HSV) và mượt hóa bằng bộ đệm lịch sử. Khối `ViolationManager` sẽ đóng gói JSON nếu xe thỏa mãn 4 điều kiện khắt khe: (1) Đèn RED, (2) Đi đúng hướng, (3) Cắt vạch dừng (Cross-product âm), và (4) Không nằm trong đa giác rẽ phải.
4. **Nhánh B - Điều khiển thích ứng:** Dùng hàm `cv2.pointPolygonTest()` để đếm xe trong vùng *Queue ROI*. Trích xuất vector đặc trưng 5 chiều (lưu lượng, vận tốc, hệ số PCU...) đưa qua SVM để nội suy mức ùn tắc. Từ đó, `AdaptiveLightController` phát lệnh can thiệp pha đèn.
5. **Trực quan & Kết xuất (Visualizer & Output):** Lớp Overlay UI vẽ bounding box, cảnh báo và bảng HUD lên ảnh. Ở chế độ `compare`, hệ thống thu phóng cục bộ và dùng `cv2.hconcat()` ghép ngang hai nhánh thành một khung hình toàn cảnh 1920×540 . Cuối cùng, `ViolationClipExtractor` xả bộ đệm để đóng gói video vi phạm, hình ảnh trực quan được minh họa tại Hình 3.

3.2.3 Cơ chế phân luồng theo ba chế độ vận hành

Để phục vụ nhiều kịch bản đánh giá khác nhau, kiến trúc phần mềm cho phép thay đổi luồng thực thi động thông qua tham số khởi tạo dòng lệnh (`-mode`). Sơ đồ cây quyết định tại Hình 1 minh họa cụ thể quá trình phân rẽ luồng dữ liệu từ điểm khởi đầu chung `VehicleTracker`.



Hình 1: Sơ đồ phân nhánh luồng dữ liệu theo ba chế độ vận hành

Dựa trên sơ đồ phân nhánh (Hình 1), các chế độ được thực thi độc lập như sau:

1. **Chế độ violation:** Kích hoạt độc quyền Nhánh A. Hệ thống tập trung tối đa tài nguyên vào bài toán phát hiện ánh sáng vật lý, nội suy logic vi phạm và cấp phát nhật ký lỗi.
2. **Chế độ adaptive:** Kích hoạt độc quyền Nhánh B. Bỏ qua logic phát hiện vi phạm, hệ thống chuyên trách phân tích mật độ tại *Queue ROI* và tính toán sự chuyển đổi của máy trạng thái đèn tín hiệu (Traffic Light State Machine).
3. **Chế độ compare:** Thực thi đa luồng tuần tự. Nhánh A phân tích đèn vật lý và truyền tín hiệu đồng bộ trạng thái khởi tạo cho Nhánh B thông qua hàm API `initialize_state()`. Hai nhánh chạy độc lập trên cùng một tập tham số môi trường và hợp nhất khung hình render cuối cùng, cung cấp cơ sở để đánh giá trực quan hiệu năng của bộ điều khiển AI so với bộ điều khiển vật lý tính.

3.3 Thiết kế chi tiết các module lõi

3.3.1 Module nhận diện và bám vết phương tiện (Vehicle Tracker)

Module `VehicleTracker` (triển khai tại `src/core/tracker.py` và `vehicle_detector.py`) đóng vai trò là khối phân tích động học sơ cấp, cung cấp dữ liệu đầu vào cho toàn bộ hệ thống.

- **Khởi tạo và Lọc đối tượng:** Mạng nơ-ron YOLOv8 được nạp bộ trọng số tiêu chuẩn. Module can thiệp vào tầng post-processing để thiết lập bộ lọc (class filter), chỉ duy trì 4 nhãn thực thể có giá trị trong giao thông đô thị từ tập dữ liệu COCO bao gồm: car (ID 2), motorcycle (ID 3), bus (ID 5) và truck (ID 7).

- **Quản trị Định danh (ByteTrack):** Kết quả phát hiện không gian được chuyển giao cho thuật toán ByteTrack. Mỗi phương tiện được cấp phát một biến trạng thái `track_id` duy nhất, bảo toàn tính nhất quán qua chuỗi khung hình.
- **Hiệu chỉnh Bao hình Hình học (Box Expansion):** Một nhược điểm của bộ phát hiện gốc là hộp bao của nhãn `motorcycle` thường chỉ tập trung vào khung gầm xe, bỏ qua người điều khiển. Để khắc phục, module triển khai thuật toán nội suy mở rộng hộp bao lên phía trên 60% theo trục tung:

$$y_{\min}^{\text{new}} = y_{\min} - 0.6 \times (y_{\max} - y_{\min}) \quad (5)$$

Kỹ thuật này đảm bảo ảnh cắt (crop) bằng chứng bắt trọn diện mạo người vi phạm.

- **Lưu vết Động học:** Tọa độ tâm của phương tiện được đẩy vào một hàng đợi `track_history`. Tập dữ liệu chuỗi thời gian này là cơ sở cốt lõi để ước lượng vector vận tốc $\vec{v}$ và kiểm định giả thuyết giao cắt vạch dừng ở các bước tiếp theo.

3.3.2 Module phân tích và định danh tín hiệu đèn (Traffic Light Detector)

Nằm tại `src/models/traffic_light_detector.py`, module này được thiết kế với kiến trúc dự phòng 3 tầng (3-tier pipeline) nhằm đảm bảo độ tin cậy tuyệt đối trước nhiễu quang học (phơi sáng, ngược sáng):

1. **Tầng 1 - Trích xuất Không gian:** Sử dụng một mạng YOLOv8 để định vị chính xác tọa độ hộp bao chứa cụm đèn giao thông.
2. **Tầng 2 - Phân loại Máy học (SVM):** Vùng ảnh crop của đèn được phân tách và trích xuất đặc trưng không gian màu HSV. Mô hình SVM tuyến tính sẽ đánh giá vector đặc trưng này để nội suy nhãn trạng thái dự kiến.
3. **Tầng 3 - Dự phòng Quang học (HSV Fallback):** Trong trường hợp mô hình SVM trả về hệ số tin cậy thấp hoặc phát sinh bất thường, hệ thống tự động chuyển đổi (fallback) sang thuật toán phân ngưỡng tĩnh trên không gian HSV thuần túy. Thuật toán đếm mật độ điểm ảnh rơi vào các dải phổ màu (Đỏ, Vàng, Xanh) để ra quyết định cường chế.

Kết quả từ các vùng đèn đơn lẻ được tổng hợp tại hàm `get_global_traffic_light_state()` thông qua cơ chế bầu chọn (voting), thiết lập một trạng thái đèn toàn cục (RED, GREEN, YELLOW, OFF) áp dụng cho toàn bộ khung hình hiện tại.

3.3.3 Module logic phát hiện vi phạm (Violation Logic)

Được cấu trúc bởi các tệp `violation_detector.py`, `violation_logic.py`, `geometry.py` và `zones.py`, module này là hạt nhân thực thi bộ quy tắc an toàn giao thông.

- **Định nghĩa Không gian Giới hạn:** Tọa độ của vạch dừng `StopLine` (đoạn thẳng AB) và vùng rẽ phải `RightTurnZone` (đa giác lồi P) được nạp tĩnh từ file cấu hình, sau đó nội suy ánh xạ trực tiếp lên ma trận ảnh.
- **Kiểm định Giao cắt (Cross Product):** Khi `Global State == RED`, hệ thống kích hoạt phép thử Tích có hướng 2D. Phương tiện bị gán cờ vi phạm nếu vector quỹ đạo di chuyển $\overrightarrow{P_{t-1}P_t}$ tạo ra giao điểm hình học thực sự với vector vạch dừng $\overrightarrow{AB}$.

- **Cơ chế Ân xá (Point-in-Polygon - PiP):** Để xử lý ngoại lệ giao thông, hàm `cv2.pointPolygonTest()` liên tục kiểm tra tọa độ tâm phương tiện. Nếu điểm này hội tụ vào vùng `RightTurnZone`, lệnh hủy vi phạm sẽ được phát đi, triệt tiêu cảnh báo sai đối với các phương tiện rẽ phải hợp lệ.
- **Máy trạng thái Giám sát:** Lớp `ViolationManager` hoạt động như một bộ điều phối độc lập, theo dõi và khóa trạng thái vi phạm theo từng `track_id`, ngăn chặn hiện tượng spam cảnh báo (ghi nhận nhiều lần cho cùng một lỗi) khi xe di chuyển chậm qua vạch.

3.3.4 Module điều khiển tín hiệu thích ứng (Adaptive Light Controller)

Được thiết kế tại `src/core/adaptive_light_controller.py`, module này mô phỏng lý thuyết điều khiển giao thông thông minh (ITS - Intelligent Transportation Systems) dựa trên thị giác máy tính.

- **Kỹ nghệ Đặc trưng Mật độ (Density Feature Engineering):** Hệ thống cô lập tập hợp phương tiện hiện diện bên trong đa giác *Queue ROI*. Một vector đặc trưng 5 chiều $X \in \mathbb{R}^5$ được tính toán theo thời gian thực bao gồm: `motorcycle_count`, `car_count`, `stopped_vehicles` (xe có vận tốc $v \approx 0$), tải trọng quy đổi `pcu_load`, và vận tốc trung bình luồng `average_speed`.
- **Nội suy Tình trạng Tắc nghẽn:** Vector X được cấp cho mô hình phân loại SVM, ánh xạ ra tập không gian trạng thái giao thông $S \in \{\text{Low, Medium, High}\}$.
- **Máy Trạng thái Hữu hạn (FSM):** Chu trình đèn được thiết lập dưới dạng một máy trạng thái 3 pha tuần tự: `GREEN` $\rightarrow$ `YELLOW` $\rightarrow$ `RED`.
- **Bộ luật Điều khiển Động (Dynamic Policies):** Logic thời gian được can thiệp trực tiếp bởi trạng thái S :
 1. *Cắt giảm hao phí:* Rút ngắn thời gian pha Xanh nếu mật độ `Low`, tránh bắt các luồng khác chờ đợi vô ích.
 2. *Khóa Xanh (Green Lock):* Duy trì và gia hạn pha Xanh nếu phát hiện `High` (ùn tắc nghiêm trọng) để giải tỏa áp lực toàn diện cho nút giao.
 3. *Xả lũ (Red Shortening):* Chủ động rút ngắn thời gian đếm ngược của pha Đỏ khi `High` xuất hiện ở chiều chờ, ngăn chặn hàng chờ phương tiện trào ngược (queue spillback) ra ngoài ngã tư trước đó.

3.4 Thiết kế module lưu trữ và kết xuất dữ liệu

Module kết xuất dữ liệu (triển khai tại `exporter.py` và `video_clip_extractor.py`) đóng vai trò là cầu nối giữa hệ thống thị giác máy tính cục bộ và hệ thống lưu trữ/quản lý giao thông tuyến trên.

3.4.1 Cấu trúc không gian hồ sơ vi phạm

Mọi sự kiện vi phạm vượt đèn đỏ được hệ thống đóng gói thành một bản ghi (record) có cấu trúc từ điển chặt chẽ. Mỗi bản ghi bao gồm các trường thông tin cốt lõi phục vụ truy xuất pháp lý:

- **vehicle_id**: Mã định danh duy nhất của phương tiện (cấp phát bởi ByteTrack).
- **vehicle_type**: Nhân phân lớp phương tiện (*car*, *motorcycle*,...).
- **bbox**: Ma trận tọa độ không gian $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$ tại thời điểm vi phạm.
- **timestamp**: Dấu thời gian hệ thống định dạng ISO 8601.
- **frame**: Chỉ số khung hình tuyệt đối trên luồng video.
- **confidence**: Hệ số tin cậy dự đoán (objectness score) của mạng YOLOv8.

3.4.2 Giao thức kết xuất đa luồng (Multi-channel Export)

Nhằm đảm bảo tính linh hoạt trong tích hợp, module **Exporter** hỗ trợ xuất dữ liệu đồng thời trên ba luồng độc lập:

1. **Lưu trữ phi cấu trúc (JSON)**: Dữ liệu được serialize và nối thêm (append) vào tệp `.json` cục bộ, phục vụ việc đọc hiểu bằng máy.
2. **Lưu trữ dạng bảng (CSV)**: Chuyển đổi bản ghi thành các hàng dữ liệu phẳng, tối ưu hóa cho các công cụ phân tích thống kê truyền thống (Excel, Pandas).
3. **Đồng bộ thời gian thực (Webhook)**: Ngay khi sự kiện phát sinh, hệ thống đóng gói payload và thực hiện một HTTP POST Request gửi thẳng đến máy chủ quản trị trung tâm, cho phép hệ thống tuyến trên phát cảnh báo theo thời gian thực.

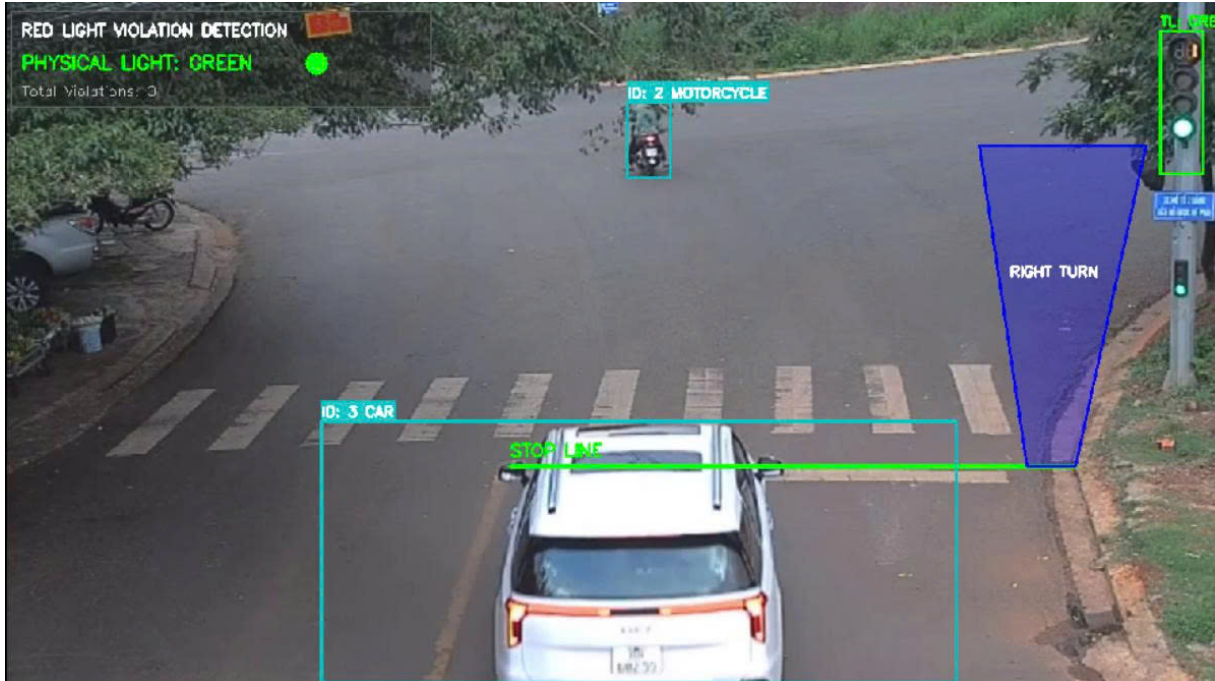
3.4.3 Trích xuất minh chứng pháp lý (ViolationClipExtractor)

Để một lỗi vi phạm có đầy đủ giá trị pháp lý, chỉ một bức ảnh chụp là không đủ. Lớp **ViolationClipExtractor** hoạt động dựa trên cơ sở của một bộ đệm vòng (Ring Buffer) liên tục lưu giữ các khung hình gần nhất. Khi cờ vi phạm được kích hoạt, module sẽ tiến hành đóng băng luồng lịch sử Δt_{before} , tiếp tục thu thập luồng tương lai Δt_{after} , và ghép nối chúng thành một tệp `.mp4` hoàn chỉnh. Cơ chế này cô lập chính xác hành vi của phương tiện từ trước khi chạm vạch cho đến khi đã vượt qua giao lộ.

3.4.4 Cơ chế thu hồi và làm sạch dữ liệu

Trong điều kiện thực tế, một phương tiện có thể cắt qua vạch dừng khi đèn đỏ nhưng lại đi vào làn ưu tiên rẽ phải (**RightTurnZone**). Để giải quyết triệt để các ca dương tính giả này, hàm `remove_violation()` được triển khai. Ngay khi logic hình học xác nhận phương tiện nằm trong đa giác rẽ phải, hàm này sẽ quét qua cấu trúc dữ liệu đang chờ ghi, tiến hành hủy bỏ bản ghi vi phạm của **vehicle_id** tương ứng trước khi tiến trình *Export* kịp thực thi, đảm bảo độ đặc hiệu (Specificity) tuyệt đối cho hệ thống.

3.5 Thiết kế giao diện trực quan (Overlay UI)



Hình 2: Giao diện trực quan hóa ở chế độ violation

Giao diện hiển thị (triển khai tại `visualization.py` và `overlay.py`) không chỉ là công cụ kiểm thử (debug) mà còn đóng vai trò là màn hình giám sát trung tâm (Head-Up Display - HUD). Hệ thống được thiết kế thành các lớp đồ họa (layers) chồng lên nhau trên ảnh gốc.

3.5.1 Không gian cảnh báo vi phạm (Violation HUD)

Được kích hoạt trong chế độ violation, lớp đồ họa này tập trung vào việc làm nổi bật hành vi sai phạm:

- **Cảnh báo thị giác động:** Khi một phương tiện vi phạm, hộp bao bounding box của phương tiện đó lập tức chuyển sang màu Đỏ và áp dụng hiệu ứng nhấp nháy (blinking effect) dựa trên hàm modulo của chỉ số khung hình, thu hút sự chú ý tức thời của người vận hành.
- **Biểu diễn hình học tĩnh:** Vạch dừng (*Stop Line*) được vẽ nét đứt và tự động đổi màu tương ứng với trạng thái đèn (Xanh/Đỏ). Vùng rẽ phải (*RightTurnZone*) được phủ một lớp đa giác bán trong suốt (alpha blending), giúp minh bạch hóa các ranh giới xử lý logic.
- **Bảng điều khiển tín hiệu:** Một HUD tĩnh đặt tại góc màn hình hiển thị trạng thái đèn hiện tại kết hợp cùng đồng hồ đếm ngược thời gian giả lập.

3.5.2 Không gian thông số mật độ (Adaptive HUD)

Kích hoạt trong chế độ adaptive, lớp đồ họa chuyển trọng tâm sang việc đánh giá lưu lượng vi mô:

- **Vùng chờ (Queue ROI):** Đa giác đo lường được vẽ đậm để phân định rõ khu vực lấy mẫu dữ liệu.
- **Bảng hiển thị động lực học:** Màn hình HUD cập nhật liên tục (real-time) các đặc trưng vừa được nội suy: số lượng xe máy, số lượng ô tô, tốc độ trung bình, và đặc biệt là hệ số tải trọng Đơn vị xe con quy đổi (PCU load).
- **Nhãn phân lớp SVM:** Kết quả đầu ra của bộ phân loại mật độ được highlight rõ ràng dưới dạng *Low*, *Medium*, hoặc *High*, đi kèm với nhãn trạng thái pha đèn tiếp theo mà máy trạng thái (State Machine) dự định chuyển đổi.

3.5.3 Giao diện đối chiếu song song (Compare Mode)

Để phục vụ mục đích trình diễn và đánh giá thuật toán, chế độ **compare** cung cấp một cấu trúc hiển thị hợp nhất. Ma trận ảnh đầu ra của hai nhánh xử lý (Violation và Adaptive) được hệ thống thu phóng (resize) và ghép nối theo chiều ngang thông qua hàm đại số ma trận `cv2.hconcat()`.

Kết quả trả về là một khung hình toàn cảnh có độ phân giải 1920×540 pixel. Bố cục side-by-side này cho phép người quan sát đối chiếu trực tiếp giữa việc "Quyết định xử phạt của hệ thống vật lý" (Bên trái) và "Lưu lượng giao thông mà bộ điều khiển AI đang phân tích" (Bên phải) ngay trên cùng một mốc thời gian thực, minh chứng rõ ràng sự tương tác của hai luồng thuật toán cốt lõi.



Hình 3: Giao diện trực quan hóa ở chế độ **compare**

4 TRIỂN KHAI HỆ THỐNG

4.1 Môi trường phần mềm và thư viện sử dụng

Hệ thống được xây dựng trên nền tảng **Windows**, sử dụng **Visual Studio Code** làm môi trường phát triển chính. Toàn bộ mã nguồn chạy trên **Python 3.10+** và được cô

lập trong môi trường ảo `venv` riêng biệt để tránh xung đột phụ thuộc với các gói cài đặt cấp hệ điều hành.

Các thư viện cốt lõi được khai báo tường minh trong tệp `requirements.txt` bao gồm:

- `ultralytics >= 8.1.0`: Cung cấp kiến trúc mạng YOLOv8 và tích hợp sẵn thuật toán ByteTrack, phục vụ bài toán phát hiện và bám vết phương tiện theo thời gian thực.
- `opencv-python >= 4.8.0.76`: Xử lý ma trận ảnh số, thực hiện phép biến đổi không gian màu từ BGR sang HSV, kiểm tra tọa độ điểm trong đa giác và kết xuất các lớp đồ họa overlay lên khung hình đầu ra.
- `scikit-learn >= 1.2.0`: Triển khai thuật toán Support Vector Machine, chuẩn hóa không gian đặc trưng bằng `StandardScaler` và sinh báo cáo đánh giá phân loại.
- `numpy >= 1.23.0` và `pandas >= 2.0.0`: Phục vụ tính toán đại số tuyến tính và xử lý dữ liệu dạng bảng `.csv`.
- `pyyaml >= 6.0.1`: Đọc và phân tích cú pháp tệp cấu hình tham số hệ thống `config.yaml`.
- `requests >= 2.31.0`: Gửi gói tin JSON chứa thông tin vi phạm đến Webhook URL bên ngoài theo phương thức HTTP POST.
- `pydantic >= 2.0.0`: Xác thực và kiểm tra kiểu dữ liệu cấu hình khi nạp từ tệp YAML vào đối tượng `SystemConfig`.

4.2 Xây dựng và tiền xử lý tập dữ liệu

4.2.1 Tập dữ liệu nhận diện đèn giao thông (YOLOv8 fine-tune)

Bộ trọng số YOLOv8 gốc được huấn luyện trên tập COCO chỉ có khả năng phát hiện vị trí của cụm đèn tín hiệu dưới nhãn `traffic light` chung, không phân biệt được các trạng thái phát sáng cụ thể. Để khắc phục điều này, đề tài thực hiện fine-tune một bộ trọng số chuyên biệt từ tập dữ liệu mở *Roboflow Traffic Light Dataset*.

Cấu trúc dữ liệu và tệp cấu hình: Không gian nhãn được thu hẹp thành 4 lớp tương ứng với 4 trạng thái thực tế của đèn tín hiệu, được định nghĩa trong `dataset.yaml` như sau:

```
names:  
  0: green  
  1: off  
  2: red  
  3: yellow
```

Tập dữ liệu được chia thành ba tập con độc lập theo tỷ lệ xấp xỉ 70/20/10: tập `train/images` dùng để cập nhật trọng số mạng qua lan truyền ngược, tập `valid/images` dùng để theo dõi hội tụ và phát hiện overfitting trong quá trình huấn luyện, và tập `test/images` dùng để đánh giá độc lập sau khi hoàn thành huấn luyện.

Trích xuất đặc trưng cho SVM (`extract_features.py`): Sau khi có tập dữ liệu ảnh, kịch bản `extract_features.py` xử lý từng bounding box trong tệp nhãn định dạng

YOLO (.txt) để cắt crop vùng đèn và trích xuất vector đặc trưng 27 chiều phục vụ huấn luyện SVM phân loại màu. Vector đặc trưng được xây dựng theo hai nhóm:

- **Đặc trưng toàn cục (9 chiều):** Giá trị trung bình 3 kênh RGB (`mean_r`, `mean_g`, `mean_b`), giá trị trung bình 3 kênh HSV (`mean_h`, `mean_s`, `mean_v`) và độ lệch chuẩn 3 kênh HSV (`std_h`, `std_s`, `std_v`) tính trên toàn bộ vùng crop đèn.
- **Đặc trưng không gian (18 chiều):** Crop đèn được chia thành 3 phân đoạn theo chiều dọc nếu $h \geq w$ (đèn đứng) hoặc theo chiều ngang nếu $h < w$ (đèn nằm ngang). Với mỗi phân đoạn, hệ thống tính 6 giá trị trung bình màu (`mean_r`, `mean_g`, `mean_b`, `mean_h`, `mean_s`, `mean_v`), cho ra $3 \times 6 = 18$ chiều đặc trưng không gian. Cách phân đoạn này giúp mô hình nhận biết vị trí bóng đèn nào đang sáng trong cụm đèn tín hiệu.

Kết quả được lưu vào `train_features.csv` và `val_features.csv`, mỗi dòng gồm 1 cột nhãn (`label`) và 27 cột đặc trưng.

Kỹ thuật tăng cường dữ liệu: Trong quá trình fine-tune YOLOv8, các phép biến đổi ngẫu nhiên được áp dụng để mô hình học được đặc trưng màu sắc cục bộ của đèn thay vì ghi nhớ bối cảnh nền, bao gồm: điều chỉnh kênh Value trong không gian HSV để mô phỏng sự chênh lệch ánh sáng giữa ban ngày và ban đêm, lật ảnh ngang (Horizontal Flip), và kỹ thuật ghép mảnh Mosaic kết hợp 4 ảnh thành một mẫu huấn luyện.

4.2.2 Tập dữ liệu mật độ giao thông (thu thập và gán nhãn tự động)

Module điều khiển đèn thích ứng cần một tập dữ liệu có giám sát để phân loại mức độ ùn tắc theo thời gian thực, nhưng việc gán nhãn thủ công từng khung hình video là không khả thi về mặt thời gian. Đề tài xây dựng kịch bản `collect_density_features.py` để tự động thu thập và gán nhãn dữ liệu từ video giao thông thực địa.

Quy trình thu thập: Kịch bản xử lý tuần tự hai video đầu vào `detection1.mp4` và `detection2.mp4`, sử dụng **VehicleTracker** (YOLOv8 + ByteTrack) để phát hiện và bám vết phương tiện. Cứ mỗi 10 frame lấy mẫu một lần — khoảng cách này được chọn để giảm tương quan giữa các mẫu liên tiếp — hệ thống lọc ra các xe có tâm bounding box nằm trong vùng Queue ROI (hình thang xác định bởi 4 đỉnh `[530, 486]`, `[1100, 486]`, `[1280, 720]`, `[380, 720]`) và trích xuất vector đặc trưng 5 chiều được mô tả trong Bảng 1.

Bảng 1: Vector đặc trưng 5 chiều dùng cho bài toán phân loại mật độ giao thông

Đặc trưng	Mô tả
<code>motorcycle_count</code>	Số lượng xe máy có tâm nằm trong vùng Queue ROI
<code>car_count</code>	Số lượng ô tô, xe buýt và xe tải trong Queue ROI
<code>stopped_vehicles</code>	Số xe dừng hẳn, xác định khi tốc độ dịch chuyển < 1.2 pixel/frame tính trên 5 frame gần nhất
<code>pcu_load</code>	Tổng tải trọng quy đổi: $n_{\text{motorcycle}} \times 0.3 + n_{\text{car}} \times 1.0$
<code>average_speed</code>	Tốc độ trung bình (pixel/frame) của tất cả xe trong ROI; mặc định là 15.0 nếu không có xe

Tốc độ từng xe được tính bằng khoảng cách Euclidean giữa vị trí hiện tại và vị trí 5 frame trước trong `track_history`, chia cho số frame:

$$v_i = \frac{\|\mathbf{p}_{\text{now}} - \mathbf{p}_{\text{now}-5}\|_2}{5} \quad (6)$$

Nếu xe chưa có đủ 5 frame lịch sử, tốc độ được đặt mặc định là 999.0 và không tính vào `stopped_vehicles` hay `average_speed`.

Luật gán nhãn tự động: Nhãn $y \in \{0, 1, 2\}$ tương ứng ba mức *Low/Empty*, *Medium/Normal* và *High/Congested* được gán theo hàm quyết định dựa trên `pcu_load` và `stopped_vehicles`:

$$y = \begin{cases} 0 \text{ (Low/Empty)}, & \text{nếu } \text{pcu_load} = 0 \\ 1 \text{ (Medium/Normal)}, & \text{nếu } \text{stopped_vehicles} = 0 \vee \text{pcu_load} \leq 0.6 \\ 2 \text{ (High/Congested)}, & \text{các trường hợp còn lại} \end{cases} \quad (7)$$

Nhãn 0 xác nhận khi ROI hoàn toàn trống. Nhãn 1 xác nhận khi xe đang lưu thông bình thường hoặc chỉ có 1–2 xe máy dừng nhẹ ($\text{PCU} \leq 0.6$). Nhãn 2 xác nhận khi có mật độ cao kèm theo xe dừng đáng kể. Kết quả toàn bộ được ghi vào `data/traffic_light/density_features.csv` với 6 cột: 5 đặc trưng đầu vào và 1 cột nhãn label.

4.3 Huấn luyện các mô hình phân loại SVM

Đề tài huấn luyện hai mô hình SVM độc lập, mỗi mô hình giải quyết một bài toán phân loại riêng biệt với không gian đặc trưng và không gian nhãn khác nhau. Việc sử dụng SVM thay cho mạng nơ-ron sâu là lựa chọn phù hợp cho các bài toán có số chiều đặc trưng thấp (5 và 27 chiều), dữ liệu huấn luyện giới hạn và yêu cầu tốc độ suy diễn cao trong vòng lặp thời gian thực.

4.3.1 SVM phân loại màu đèn giao thông (`train_svm.py`)

Dữ liệu được nạp từ `train_features.csv` và `val_features.csv`. Mỗi mẫu là một vector 27 chiều trích xuất từ `extract_features.py`, ánh xạ vào không gian nhãn 4 lớp $Y \in \{0 \text{ (green)}, 1 \text{ (off)}, 2 \text{ (red)}, 3 \text{ (yellow)}\}$.

Các bước triển khai cụ thể:

1. **Tách đặc trưng và nhãn:** Cột label được tách ra khỏi DataFrame bằng `df.drop(columns=['label'])`, phần còn lại là ma trận đặc trưng $\mathbf{X}$.
2. **Chuẩn hóa đặc trưng:** `StandardScaler` được fit trên $\mathbf{X}_{\text{train}}$ và chỉ transform trên $\mathbf{X}_{\text{val}}$, đưa toàn bộ đặc trưng về phân phối chuẩn ($\mu = 0, \sigma = 1$). Bước này ngăn các đặc trưng có phương sai lớn — chẳng hạn giá trị kênh Value có biên độ 0–255 — áp đảo các đặc trưng có biên độ nhỏ hơn.
3. **Khởi tạo và huấn luyện:**

```
svm = SVC(kernel='rbf', C=1.0, class_weight='balanced',
           probability=True, random_state=42)
svm.fit(X_train_scaled, y_train)
```

Tham số `class_weight='balanced'` tự động điều chỉnh trọng số mỗi lớp tỷ lệ nghịch với tần suất xuất hiện, xử lý trường hợp lớp `off` xuất hiện ít hơn đáng kể so với `red` và `green` trong điều kiện ban ngày. Tham số `probability=True` cho phép truy xuất xác suất tin cậy của từng lớp trong quá trình suy diễn.

4. **Đánh giá:** Tính Accuracy, in Classification Report và Confusion Matrix trên tập `val`.
5. **Lưu mô hình:** Cặp `{scaler, svm, classes}` được đóng gói vào một dict và tuần tự hóa bằng `pickle` ra tệp `src/models/svm_traffic_light.pkl` để nạp trực tiếp vào pipeline suy diễn.

4.3.2 SVM phân loại mật độ giao thông (`train_density_svm.py`)

Dữ liệu đầu vào là `density_features.csv` với vector đặc trưng 5 chiều đã mô tả ở mục 4.2.2, nhãn đầu ra gồm 3 mức mật độ $Y \in \{0 \text{ (Low/Empty)}, 1 \text{ (Medium/Normal)}, 2 \text{ (High/Congested)}\}$. Điểm khác biệt so với mô hình màu đèn:

- **Phân chia tập dữ liệu:** Do không có tệp `val` riêng, tập dữ liệu được chia tự động theo tỉ lệ 80/20 bằng `train_test_split(test_size=0.2, stratify=y, random_state=42)`. Đối số `stratify=y` đảm bảo phân phối nhãn được duy trì đồng đều giữa tập `train` và tập `val`, tránh trường hợp tập `val` không có mẫu lớp `High`.
- **Tham số huấn luyện:**

```
svm = SVC(kernel='rbf', C=2.0, class_weight='balanced',
           probability=True, random_state=42)
```

Hệ số phạt `C=2.0` cao hơn mô hình màu đèn (`C=1.0`) để tăng khả năng phân tách đường biên với không gian đặc trưng mật độ phức tạp và dữ liệu thực địa có nhiều nhiễu hơn.

- **Lưu mô hình:** Kết quả được lưu vào `src/models/svm_traffic_density.pkl` theo cùng cấu trúc dict `{scaler, svm, classes}`.

4.4 Cài đặt pipeline xử lý chính (`main.py`)

Module `main.py` điều phối toàn bộ vòng đời hệ thống thông qua hàm `run_pipeline()`. Khung chương trình hỗ trợ 3 chế độ vận hành (truyền qua đối số `-mode`):

- `violation`: Giám sát và bắt lỗi vượt đèn đỏ.
- `adaptive`: Điều khiển pha đèn thích ứng bằng SVM phân loại mật độ.
- `compare`: Kích hoạt song song cả hai tiến trình để đối chiếu trực quan.

4.4.1 Cấu trúc luồng xử lý (Main Loop)

Sau khi nạp cấu hình từ `SystemConfig` và khởi tạo các module tính để tối ưu RAM/VRAM, vòng lặp xử lý từng khung hình (frame) được thực thi qua 5 pha đồng bộ:

1. **Pha Tracking:** `VehicleTracker` cấp phát ID. Riêng nhãn `motorcycle`, hộp bao (bounding box) tự động giãn nở lên trên 60% chiều cao nhằm bắt trọn vùng thân người lái phục vụ trích xuất bằng chứng:

$$y_1^{\text{new}} = \max(0, y_1 - 0.6 \times (y_2 - y_1)) \quad (8)$$

2. **Pha Detection:** `TrafficLightDetector` nhận diện cụm đèn và tổng hợp thành trạng thái phát sáng toàn cục (RED/GREEN/YELLOW).

3. **Pha Logic phân nhánh:**

- *Logic Vi phạm:* `ViolationDetector` kiểm tra quỹ đạo cắt vạch. Nếu phát hiện, kích hoạt ghi JSON, gửi Webhook và lệnh cắt video. Nếu xe lọt vào `right_turn_zone`, cờ vi phạm lập tức bị hủy bỏ.
- *Logic Thích ứng:* Lọc phương tiện trong `Queue ROI`, tính hệ số `pcu_load` và đưa vào SVM nội suy mật độ, từ đó `AdaptiveLightController` trả về quyết định can thiệp (ví dụ: cờ Green Lock).

4. **Pha Visualization:** Dựng lớp Overlay UI (HUD, Bounding Box, Polygon) lên ma trận ảnh.

5. **Pha I/O:** Xuất khung hình xuống bộ đệm. Đặc biệt ở chế độ `compare`, hệ thống dùng `cv2.hconcat` ghép ngang hai luồng xử lý thành một khung hình toàn cảnh kích thước 1920×540 .

4.4.2 Cơ chế kết xuất bằng chứng (ViolationClipExtractor)

Để bảo toàn tính pháp lý, hệ thống triển khai một bộ đệm vòng (Ring Buffer) dạng deque lưu giữ ≈ 1 giây video trong quá khứ (`clip_before_sec: 1`).

Ngay khi hàm `trigger_violation()` được gọi, luồng I/O sẽ sao chép bộ đệm quá khứ, tiếp tục ghi thêm 2 giây tương lai (`clip_after_sec: 2`), và đóng gói thành tệp `.mp4` hoàn chỉnh tại thư mục `Luutru_Vipham`. Nếu logic ghi nhận phương tiện rẽ phải hợp lệ, lệnh `cancel_clip()` sẽ can thiệp để hủy bỏ tiến trình ghi đĩa, tiết kiệm tài nguyên lưu trữ.

4.5 Thiết kế cấu trúc tham số toàn cục (config.yaml)

Tuân thủ nguyên lý Tách biệt Mối quan tâm (Separation of Concerns), toàn bộ tham số tính và siêu tham số được quản lý tập trung tại tệp `config.yaml`, chia thành 4 phân vùng chính:

- **Trọng số và Bộ lọc (Models & Thresholds):** Cấu hình đường dẫn tệp `yolov8s.pt`. Thiết lập ngưỡng tin cậy bắt xe (0.25), bắt đèn (0.05) và ngưỡng lọc chồng lấp NMS IoU (0.45).
- **Hình học không gian (Scene):** Chứa các ma trận tọa độ định tuyến thuật toán:

- `stop_line`: Khai báo tọa độ vạch dừng ngang, ví dụ [533, 486, 1126, 486].
 - `right_turn_zone`: Đa giác 4 đỉnh khoanh vùng rẽ phải để loại bỏ nhiễu.
 - `static_traffic_lights`: Tọa độ cứng dự phòng khi module quang học bị che khuất.
 - `target_resolution`: Tự động ép kiểu (resize) video đầu vào về chuẩn 1280 × 720.
- **Cấu hình I/O (Output)**: Quản trị cờ lưu trữ video, đường dẫn JSON log và địa chỉ Webhook URL phục vụ đồng bộ dữ liệu đám mây.
 - **Lưu trữ pháp lý (Storage)**: Định mức thời gian trích xuất `ViolationClipExtractor` (1s trước, 2s sau vi phạm) và ảnh chụp toàn cảnh (scene frame).

Cấu trúc phi tập trung này đảm bảo khả năng triển khai hệ thống (cross-deployment) lên hàng loạt giao lộ vật lý khác nhau mà không yêu cầu kỹ sư phải biên dịch lại mã nguồn Python.

5 Đánh Giá

5.1 Kết quả nhận diện đèn giao thông

5.1.1 Độ chính xác của SVM phân loại màu đèn trên tập kiểm thử

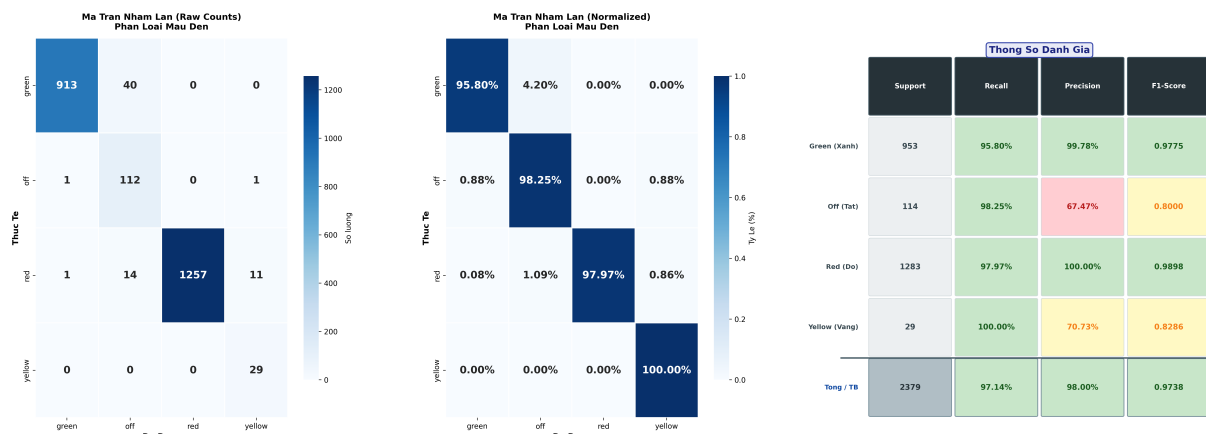
Mô hình SVM phân loại trạng thái đèn giao thông được huấn luyện với kernel RBF, tham số $C = 1,0$ và `class_weight='balanced'` nhằm xử lý mất cân bằng dữ liệu giữa các lớp. Trên tập `val_features.csv` gồm 2379 mẫu, mô hình phân loại đúng 2311 mẫu, đạt **weighted F1-score = 0,9738** và độ chính xác tổng thể **97,14%**. Kết quả chi tiết theo từng lớp được trình bày trong Bảng 2.

Bảng 2: Kết quả phân loại theo lớp của SVM nhận diện đèn giao thông

Lớp	Support	Recall	Precision	F1-Score
green	953	95,80%	99,78%	0,9775
off	114	98,25%	67,47%	0,8000
red	1283	97,97%	100,00%	0,9898
yellow	29	100,00%	70,73%	0,8286
Tổng / TB	2379	97,14%	98,00%	0,9738

Lớp **red** — có vai trò quyết định trực tiếp đối với module phát hiện vi phạm — đạt Precision **100%** và F1-score **0,9898** trên tập mẫu lớn nhất (1283 mẫu). Điều này có nghĩa mô hình **không bao giờ nhận nhầm trạng thái khác thành đèn đỏ**, loại bỏ hoàn toàn nguy cơ phát sinh vi phạm ảo do lỗi nhận diện đèn.

Hai lớp **off** và **yellow** có Precision thấp hơn (lần lượt 67,47% và 70,73%), nguyên nhân chủ yếu là **mất cân bằng dữ liệu**: lớp **yellow** chỉ có 29 mẫu vì đèn vàng xuất hiện rất ngắn trong mỗi chu kỳ đèn, lớp **off** có 114 mẫu so với 1283 mẫu của lớp **red**. Tuy nhiên, cả hai lớp đều đạt Recall cao (98,25% và 100,00%), cho thấy mô hình không bỏ sót các trạng thái này.



Hình 4: Ma trận nhầm lẫn của SVM phân loại trạng thái đèn giao thông trên tập kiểm thử

5.1.2 Phân tích ma trận nhầm lẫn

Hình 4 trình bày ma trận nhầm lẫn dạng số lượng (Raw Counts) và dạng tỉ lệ phần trăm (Normalized) của mô hình. Phần lớn mẫu tập trung trên đường chéo chính, phản ánh kết quả phân loại tổng thể ổn định. Phân tích các nhầm lẫn cụ thể:

- **green → off (40 mẫu, 4,20%):** Đây là nhầm lẫn lớn nhất trong toàn bộ ma trận. Nguyên nhân là đèn xanh có cường độ sáng thấp (do camera ở xa, góc nghiêng hoặc thời tiết) có đặc trưng HSV gần với trạng thái tắt, khiến SVM khó phân biệt hai lớp này.
- **red → off (14 mẫu, 1,09%) và red → yellow (11 mẫu, 0,86%):** Một số mẫu đèn đỏ có sắc lệch về vàng hoặc độ sáng thấp bị phân loại nhầm. Đáng chú ý là chỉ có **1 mẫu red bị nhầm thành green (0,08%)**, hạn chế tối đa rủi ro đèn đỏ bị nhận thành đèn xanh — trường hợp nguy hiểm nhất trong bài toán phát hiện vi phạm.
- **off và yellow:** Hai lớp này có rất ít nhầm lẫn tuyệt đối (mỗi lớp chỉ 1–2 mẫu bị nhầm sang lớp khác), tuy nhiên Precision thấp do số lượng mẫu nhỏ khiến tỉ lệ phần trăm bị kéo xuống. Lớp **yellow** đạt Recall 100% — không bỏ sót bất kỳ mẫu đèn vàng nào.

5.1.3 So sánh SVM với phương pháp dự phòng HSV (fallback)

Bên cạnh SVM là phương pháp chính, hệ thống tích hợp thêm bộ phân loại dự phòng thuần HSV (`_classify_state_hsv`), được kích hoạt khi SVM trả về kết quả UNKNOWN hoặc gặp lỗi trong quá trình suy luận. Bảng 3 tóm tắt sự khác biệt giữa hai phương pháp.

Cơ chế kết hợp hai phương pháp giúp hệ thống luôn trả về trạng thái đèn hợp lệ, tránh trường hợp UNKNOWN làm gián đoạn pipeline phát hiện vi phạm. Ngoài ra, hệ thống còn áp dụng *temporal smoothing* (làm mịn theo thời gian) và *hybrid correction* riêng cho đèn vàng, bổ sung thêm một lớp ổn định hóa kết quả phân loại trong các frame liên tiếp.

Bảng 3: So sánh SVM và phương pháp dự phòng HSV

Tiêu chí	SVM (chính)	HSV Fallback (dự phòng)
Độ chính xác	97,14%	Thấp hơn, phụ thuộc ngưỡng
Cần dữ liệu huấn luyện	Có	Không
Nhạy với ánh sáng	Thấp (đã học)	Cao (ngưỡng cố định)
Tốc độ xử lý	Nhanh	Rất nhanh
Vai trò	Phương pháp chính	Dự phòng tránh UNKNOWN

5.2 Kết quả phát hiện vi phạm

5.2.1 Trường hợp vi phạm trên video kiểm thử

Để đánh giá module phát hiện vi phạm, kết quả xuất ra từ tệp `violations.json` (chế độ violation) được đối chiếu thủ công với nội dung video kiểm thử. Mỗi bản ghi vi phạm trong tệp JSON gồm các trường `vehicle_id`, `vehicle_type`, `frame`, `timestamp`, `confidence`, `bbox` và `scene_path`, là cơ sở để xác định thời điểm và phương tiện được hệ thống ghi nhận vi phạm. Ví dụ một bản ghi vi phạm thực tế được hệ thống xuất ra như sau:

```
{
  "vehicle_id": 34,
  "vehicle_type": "motorcycle",
  "frame": 1310,
  "timestamp": 1781320850.735,
  "datetime": "2026-06-13 10:20:50",
  "confidence": 0.7533,
  "bbox": [900, 297, 983, 486],
  "crop_path": "",
  "scene_path": "Luutru_Vipham\\2026-06-13\\Vuot_Den_Do\\
    VehicleID_34_motorcycle_F01310_20260613_102050..."
}
```

Hình 5 minh họa khung cảnh tại thời điểm hệ thống ghi nhận vi phạm trên: đèn giao thông đang ở trạng thái đỏ, phương tiện mang ID 34 (xe máy) đã đi qua vạch dừng trong khi các phương tiện khác vẫn đang dừng chờ đèn, được hệ thống khung đỏ và hiển thị nhãn cảnh báo VIOLATION #34 MOTORCYCLE, kèm ảnh cận cảnh phương tiện được chèn ở góc trên-trái khung hình.

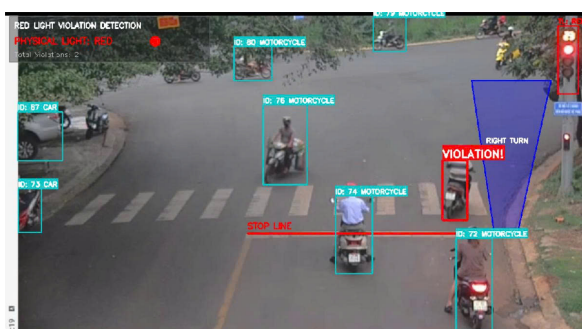
5.2.2 Phân tích các trường hợp hủy vi phạm rẽ phải

Bên cạnh việc ghi nhận vi phạm, `ViolationManager` còn thực hiện cơ chế hủy vi phạm khi phương tiện đi vào vùng cho phép rẽ phải (`RightTurnZone`) sau khi đã bị ghi nhận là vượt vạch dừng. Khi đó, hệ thống gọi `exporter.remove_violation()` để xóa bản ghi tương ứng khỏi `violations.json` và `clip_extractor.cancel_clip()` để hủy clip đang thu. Việc đánh giá cơ chế này được thực hiện bằng cách đối chiếu các trường hợp hủy vi phạm (được in ra dưới dạng log `[CANCEL] Vehicle ID #... entered right-turn zone`) với hành vi thực tế của phương tiện trong video, chia thành hai trường hợp:



Hình 5: Khung hình minh họa một trường hợp phát hiện vi phạm

- **Hủy đúng:** phương tiện ban đầu bị ghi nhận do cắt qua vạch dừng khi đèn đỏ, sau đó thực sự rẽ phải vào làn đường cho phép — việc hủy vi phạm trong trường hợp này là chính xác, phản ánh đúng quy định rẽ phải khi đèn đỏ.
- **Hủy sai:** phương tiện cắt qua vạch dừng và tiếp tục di chuyển thẳng hoặc rẽ trái (vi phạm thực sự), nhưng quỹ đạo di chuyển đi qua vùng **RightTurnZone** (do vùng này được định nghĩa là một đa giác cố định, có thể trùng một phần với làn đi thẳng/rẽ trái ở một số góc camera), khiến vi phạm bị hủy nhầm.



(a) Phương tiện cắt vạch dừng (Stop Line) bị gắn cờ "VIOLATION!"



(b) Phương tiện đi vào vùng đa giác rẽ phải (Right Turn Zone) không bị xử phạt

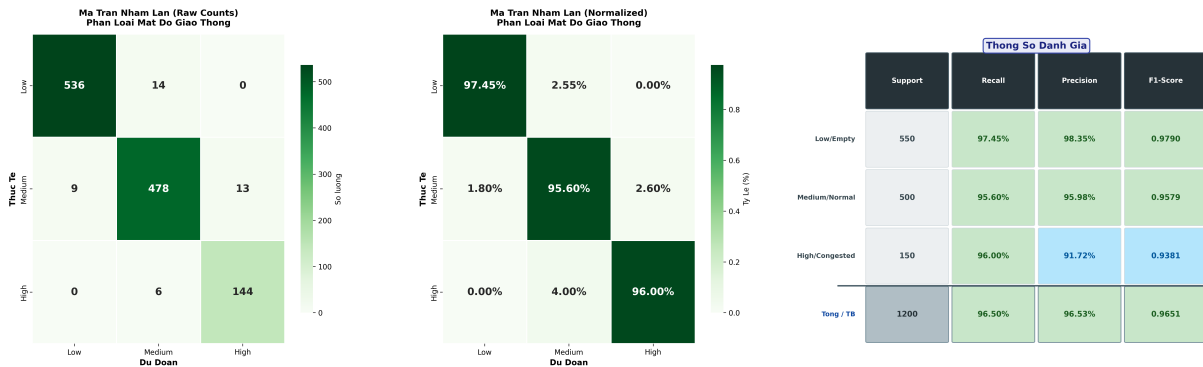
Hình 6: Kết quả thực nghiệm phân tích quỹ đạo động học: Đối chiếu giữa hành vi vi phạm vượt vạch dừng (a) và hành vi rẽ phải hợp lệ (b) trong chu kỳ đèn Đỏ

5.3 Kết quả module điều khiển tín hiệu thích ứng

Module điều khiển thích ứng (Adaptive Controller) kết hợp khả năng nhận diện mật độ giao thông và tự động điều chỉnh chu kỳ đèn nhằm giảm thiểu ùn tắc vi mô tại nút giao.

5.3.1 Đánh giá phân loại mật độ dòng xe bằng SVM

Mô hình SVM phân loại mật độ tại vùng chờ (*Queue ROI*) được đánh giá trên tập dữ liệu gồm 1200 mẫu thuộc ba lớp: Low/Empty, Medium/Normal và High/Congested. Kết quả chi tiết được trình bày tại Hình 7 và Bảng 4.



Hình 7: Ma trận nhầm lẫn của SVM phân loại mật độ giao thông trên tập kiểm thử

Bảng 4: Kết quả phân loại theo lớp của SVM phân loại mật độ giao thông

Lớp	Support	Recall	Precision	F1-Score
Low/Empty	550	97,45%	98,35%	0,9790
Medium/Normal	500	95,60%	95,98%	0,9579
High/Congested	150	96,00%	91,72%	0,9381
Tổng / TB	1200	96,50%	96,53%	0,9651

Mô hình đạt weighted F1-score **0,9651** và độ chính xác tổng thể **96,50%**, cho thấy kết quả ổn định trên cả ba lớp mật độ. Một điểm đáng chú ý là **không có mẫu nào nhầm lẫn chéo giữa Low và High** (và ngược lại), phản ánh đúng thực tế: hai trạng thái giao thông cực đoan này có đặc trưng PCU và tốc độ trung bình rất khác biệt, dễ phân biệt hơn với lớp trung gian.

Nhầm lẫn chủ yếu xảy ra tại ranh giới Medium ↔ High (13 mẫu Medium bị dự đoán thành High, chiếm 2,60%), do ranh giới giữa hai mức mật độ này trong thực tế không có ngưỡng cứng rõ ràng. Về mặt điều khiển, việc nhận nhầm Medium thành High sẽ kích hoạt chính sách *Green Lock* — kéo dài đèn xanh để giải tỏa phương tiện — đây là hướng sai số an toàn, không gây ùn tắc dây chuyền.

5.3.2 Hoạt động của máy trạng thái điều khiển (State Machine)

Dựa trên nhãn mật độ đầu ra của SVM, hệ thống tự động kích hoạt chính sách can thiệp chu kỳ đèn tương ứng, được tóm tắt trong Bảng 5.

Bảng 5: Các chính sách điều khiển tín hiệu thích ứng của hệ thống

Nhãn mật độ	Chính sách	Tác động
Low	Green Shortening (Rút ngắn đèn xanh)	Cắt giảm thời gian đèn xanh dư thừa, giảm thời gian chờ cho luồng xe cắt ngang.
Medium	Fixed-time (Chu kỳ mặc định)	Không can thiệp, duy trì nhịp lưu thông tiêu chuẩn của nút giao.
High	Green Lock (Khóa đèn xanh)	Kéo dài đèn xanh để giải tỏa phương tiện dồn ứ, chống hiện tượng trào ngược hàng chờ (<i>queue spillback</i>).

5.3.3 Hiệu năng chế độ so sánh song song (Compare Mode)

Chế độ **compare** ghép song song hai luồng xử lý trên cùng một khung hình (1920×540 pixel): nhánh trái chạy module phát hiện vi phạm (kiểm tra giao cắt vạch dừng bằng cross product), nhánh phải chạy module điều khiển thích ứng (nội suy mật độ trong Queue ROI). Hệ thống duy trì FPS ổn định khi xử lý đồng thời cả hai thuật toán mà không xảy ra tràn bộ nhớ, cho thấy pipeline đa luồng được quản lý tài nguyên hiệu quả.

6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề tài đã hoàn thành mục tiêu xây dựng một hệ thống giám sát giao thông khép kín (End-to-End). Bằng việc tích hợp mạng nơ-ron YOLOv8, thuật toán bám vết đa đối tượng ByteTrack, và các mô hình Học máy Vectơ Hỗ trợ (SVM), hệ thống không chỉ giải quyết thành công bài toán tự động hóa phát hiện vi phạm vượt đèn đỏ mà còn mở rộng sang lĩnh vực điều phối luồng giao thông thông minh thông qua cơ chế can thiệp pha đèn thích ứng.

6.2 Những kết quả đạt được

- **Phát hiện vi phạm độ tin cậy cao:** Triệt tiêu tối đa tỷ lệ Dương tính giả (False Positive) nhờ kết hợp chặt chẽ giữa thuật toán Tích có hướng (nhận diện cắt vạch) và Point-in-Polygon (ân xá phương tiện rẽ phải).
- **Điều khiển thích ứng (Adaptive Controller):** Mô phỏng thành công cơ chế điều tiết chu kỳ đèn theo thời gian thực dựa trên đặc trưng mật độ vi mô, tối ưu hóa năng lực giải phóng mặt đường của nút giao.
- **Kiến trúc phần mềm linh hoạt (Modular Pipeline):** Toàn bộ tham số cấu hình được tách bạch, mã nguồn chia tách thành các phân hệ độc lập giúp hệ thống dễ dàng bảo trì và triển khai chéo (cross-deployment).

6.3 Hạn chế của hệ thống

- **Khả năng tích hợp luồng trực tiếp:** Hệ thống hiện được tối ưu để phân tích dữ liệu video ngoại tuyến (offline video), chưa hỗ trợ giao thức truyền phát thời gian thực (RTSP) từ IP Camera.
- **Phụ thuộc góc phối cảnh:** Độ chính xác của thuật toán đo lường quỹ đạo bị ảnh hưởng mạnh bởi góc ngẩng (pitch angle) và độ phân giải đầu vào của camera vật lý.
- **Nhiều quang học:** Hiệu năng nội suy trạng thái đèn bị suy giảm trong điều kiện thiếu sáng trầm trọng hoặc khi cụm đèn bị che khuất bởi các phương tiện trọng tải lớn.

6.4 Hướng phát triển tương lai

Để hoàn thiện và thương mại hóa sản phẩm, các hướng nghiên cứu tiếp theo bao gồm:

1. **Tích hợp RTSP & IP Camera:** Mở rộng pipeline đường ống I/O để trực tiếp tiếp nhận và giải mã luồng video thời gian thực từ các camera giao thông hiện hành.
2. **Nâng cấp thuật toán bám vết:** Tích hợp kỹ thuật Nhận dạng lại (Re-Identification / Re-ID) vào ByteTrack để triệt tiêu hoàn toàn lỗi đứt gãy định danh (ID switching) khi phương tiện bị che khuất.
3. **Mở rộng hệ sinh thái vi phạm:** Đào tạo thêm các hành vi vi phạm thứ cấp như: lấn làn, đi sai hướng, hoặc đo lường tốc độ (Speed Estimation).
4. **Triển khai điện toán biên (Edge Computing):** Tối ưu hóa tensor và nhúng thuật toán lên các nền tảng thiết bị phần cứng chuyên dụng (như NVIDIA Jetson Nano/Orin) nhằm xử lý trực tiếp tại tủ điều khiển giao thông.
5. **Xây dựng Web Dashboard:** Phát triển nền tảng giao diện web đám mây để trực quan hóa biểu đồ thống kê mật độ và quản lý truy xuất nhật ký vi phạm pháp lý.

TÀI LIỆU THAM KHẢO

- [1] Ultralytics, *YOLOv8: The State-of-the-Art YOLO Model*, [Online]. Available: <https://github.com/ultralytics/ultralytics>, 2023.
- [2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-Object Tracking by Associating Every Detection Box,” *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [3] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

PHÂN CÔNG NHIỆM VỤ

Bảng 6: Phân công nhiệm vụ các thành viên trong nhóm

STT	Họ và tên	MSSV	Vai trò & Nhiệm vụ
1	Nguyễn Cảnh Hào	24022319	AI Core Engineer Fine-tune YOLOv8, huấn luyện SVM màu đèn, xây dựng module nhận diện và bám vết phương tiện
2	Nguyễn Duy Hưng	24022349	Logic Developer Cài đặt Cross Product, Point-in-Polygon, xây dựng toàn bộ logic phát hiện vi phạm
3	Nguyễn Duy Khánh	24022361	Storage & Testing Engineer Xây dựng module lưu trữ bằng chứng, chạy kiểm thử và thống kê kết quả thực nghiệm
4	Lương Quang Hoàng	24022337	Technical Writer Thu thập hình ảnh, sơ đồ, code,... chủ trì biên soạn báo cáo